

CAPRM-Flood: A Reproducible C++/Python Framework for Property-Level Flood Exposure Indexing

Sterling Alexander
M.S. Computer Science
Rochester Institute of Technology
Rochester, New York, USA
Advisor: Professor Minseok Kwon

I. INTRODUCTION

Flooding is the most common weather-related natural disaster [1]. In the northeastern United States, climate change is increasing exposure to flooding, partly through increases in heavy precipitation [2]. Flooding can damage buildings, homes, roadways and water and electrical infrastructure [2]. FEMA estimates that just one inch of flooding can cost up to approximately \$25,000 in damage to a home [3]. Monroe County has been included in 4 FEMA flood-related disaster or emergency declarations from 1956–2022, the most recent of which took place in 2017; further, the Monroe County hazard plan identifies multiple local mechanisms increasing flood susceptibility, including lakeshore flooding and erosion along Lake Ontario, riverine flooding, urban drainage flooding, and flash flooding. FEMA reports that each dollar spent on mitigation saves two dollars in post-flood recovery costs [4].

Property-level flood analysis uses geospatial data detailing mapped flood hazards, terrain, hydrography, precipitation and property location to assess vulnerability and predict loss. FEMA maintains the National Flood Hazard Layer as a database containing geospatial data for flood hazard prediction, including both flood classification and level of risk [5]. The National Flood Insurance Program (NFIP) uses the FEMA Flood Map Service Center (MSC) as its official source for flood hazard information [6]. Beyond FEMA’s MSC, the New York State GIS Program Office provides parcel-centroid data for all 62 New York counties; centroids are generated from the state’s parcel-map program and are mapped to assessment-roll data [7]. When using multiple data sources like these simultaneously, construction of a property-level feature table requires the reconciliation of data sources comprising different schemas, geometry types, coordinate reference systems, and file formats. A spatial join combines observations by geometric relationships instead of a shared identifier or key [8].

Across modern spatial analytics systems driven by spatial joins, research has observed significant variations in performance, demonstrating the need for explicit benchmarking over assumptions about equivalent performance [9]. Established libraries facilitate geospatial feature extraction: GeoPandas

allows for planar spatial operations driven by predicates, such as `intersects` [10]. Because GeoPandas performs planar operations, its geometric and distance calculations depend on appropriate coordinate reference system selection. Shapely offers distinct topological predicates: `contains` excludes points that lie only on a polygon boundary, whereas `covers` includes geometries lying within either the interior or boundary of the selected geometry [11]. A common structure across indexed spatial processing systems is the filter-and-refine structure, which first searches bounding boxes and then applies a predicate to filter candidate geometries [12]. Boundary-point results may differ under different predicate semantics or across different spatial analytics libraries; in general, large batches benefit from spatial indexes that reduce the number of exact comparisons needed to identify matching geometry. These traits suggest the need for separate evaluations of performance and correctness in a spatial feature-extraction system. This project seeks to answer the following question: Can a reproducible C++/Python pipeline extract property-level flood-hazard features from public geospatial data while preserving agreement with a trusted Python baseline and providing competitive large-batch spatial-processing performance?

This paper presents CAPRM-Flood, a benchmarked spatial feature-extraction pipeline for property-to-hazard joins. The study region is Monroe County and Rochester, New York. The system takes as input a stable property identifier and coordinates and outputs traceable per-property evidence fields, normalized components, and a preliminary relative exposure index. The system comprises two layers of processing driven respectively by Python and C++. Python responsibilities include ingestion, schema cleaning, normalization, GeoPandas/Shapely reference computation, validation and reporting; C++ responsibilities include indexed batch point-in-polygon processing and relevant flood-risk feature lookups, such as distance to the nearest mapped water source.

CAPRM-Flood presents a reproducible workflow for deriving traceable property-level features from public property and flood-hazard data. It provides a C++ spatial-processing core for batch property-to-hazard lookups and a validation framework for comparing its correctness and performance

against GeoPandas and Shapely, including point-in-polygon agreement and boundary-case analysis. Finally, it provides a performance evaluation framework with distinct benchmarks for ingestion, index construction, computation time, throughput, and scaling. These components provide transparent screening and feature infrastructure for reproducible research and the preparation of validated data for later risk analysis or modeling.

II. RELATED WORK

Works related to CAPRM-Flood can be grouped according to function. As FEMA’s database of current effective flood-hazard data, the NFHL provides flood-hazard geometries and regulatory mapping information derived from effective flood maps and Letters of Map Change (LOMC) [5]. FEMA’s National Risk Index offers baseline risk measurements for flooding among 17 other natural hazards at the community level, and is accessible to the public through the Resilience Analysis and Planning Tool (RAPT) [13]. Additionally, FEMA provides Hazus as a standardized set of hazard-estimation tools and corresponding data; Monroe County’s 2023 Hazard Mitigation Plan (HMP) utilizes Hazus to predict loss and damages due to flooding [4], [14]. Hazus flood-model methodology is implemented in FAST to swiftly analyze building-level flood risk [15]. These resources supply hazard information or perform broader risk analysis, whereas CAPRM-Flood investigates reproducible computations of property-level features with transparency in both feature derivation and performance evaluation.

Multi-Criteria Decision Analysis (MCDA) provides an established set of methodologies for assessing flood risk. In particular, the Analytic Hierarchy Process (AHP) facilitates the mapping of relative weights to different geospatial criteria [16]. GIS-based multi-criteria analysis has been used to map flood vulnerability in the trans-boundary Shatt Al Arab basin (Iraq-Iran) [17]. Factors used in MCDA studies include slope, elevation, land use, river proximity, drainage density, rainfall and soil type, while AHP facilitates a structured comparison of each criterion’s influence on flood vulnerability [18]. CAPRM-Flood uses explicit component weights to produce a preliminary countywide exposure index, while separately evaluating implementation correctness and computational performance.

CAPRM-Flood evaluates correctness using GeoPandas and Shapely as a Python reference environment. GeoPandas provides predicate-driven spatial joins and relies on Shapely for performing geometric operations [10]. Existing tools facilitate spatial feature extraction in C++, though prior systems research demonstrates significant performance variation across spatial systems and workloads [9]. Boost.Geometry’s R-tree implementation and spatial query interface are feasible components of a C++ core for executing indexed spatial queries [19]. In leveraging these resources, CAPRM-Flood contributes a bounded case study involving property points and real FEMA polygon geometries.

III. SYSTEM ARCHITECTURE

CAPRM-Flood implements a staged geospatial evidence pipeline, with distinct stages for ingestion of public data, computation of results via a trusted reference methodology, independent spatial computation, results validation, evidence generation, and preliminary indexing. The architecture is structured to preserve auditable property-level evidence, validate independent computations, use coordinate reference system (CRS) normalization in support of the spatial operation performed, and separate evidence and scoring processes.

The architecture’s first stage performs Python ingestion and subsequent caching: property points are loaded or retrieved, along with FEMA flood-hazard polygons, county-boundary geometry, hydrography features, and terrain raster data. For standardization purposes, raw data is normalized into standard schemas, source identifiers are recorded, operation-specific CRS values are validated across relevant data sources, and artifact manifests and checksums corresponding to the target dataset are cached for reproducibility and future data monitoring. In total, this architecture uses a Python environment to perform data access, CRS conversion, metadata sampling, reference calculations, results validation across software environments, and evidence presentation.

The dataset surveyed geographically spans Monroe County, using canonical property identifiers derived from NYS parcel centroid data. This workload represents each canonical property identifier as a unique row, facilitating the consolidation of all evidence features under the same per-property key.

The first property feature surveyed for CAPRM assessment is FEMA flood-zone membership including a Special Flood Hazard Area (SFHA) designation for each property point. A Python reference implementation collects data for this feature family by projecting property points and FEMA polygons into a shared CRS; a spatial membership predicate is subsequently applied via GeoPandas/Shapely. FEMA features referenced by this reference implementation are flattened into polygon parts and rings for a C++ point-in-polygon membership computation, including hole handling and feature-specific bounding box filtering. Evidence validated between code environments for each unique property point includes polygon matching verification, FEMA flood-zone membership label, SFHA flag, and source feature identifier and index.

CAPRM uses nearest-water distance as its second evidence family. The reference computation in Python projects both property points and US Geological Survey hydrography feature data to EPSG:26918 as a shared CRS, builds a spatial index, computes point-to-feature distances, and records feature and identification data corresponding to each point’s selected nearest hydrography feature. C++ performs two operations over this same evidence family: first, a brute-force implementation of the methodology executed in Python; second, an indexed feature-level bounding-volume hierarchy. While both C++ implementations share geometry semantics, the indexed search strategy prunes feature bounding box queries that cannot improve or tie the active result for nearest feature.

Terrain-derived evidence is used as the third risk assessment feature. Because this implementation layer depends on raster-window sampling and raster reprojection instead of vector geometry operations, it is currently implemented entirely in Python. A source digital elevation model (DEM) is reprojected to EPSG:26918 to enable usage of meter-based coordinates. At each property centroid, data is recorded for elevation, local mean elevation, relative elevation, slope in degrees, sampling radius, and terrain CRS. To preserve the traceability and independent analysis of the DEM data, the terrain evidence is recorded as a separate evidence table and manifest with a raster checksum separate from the integrated FEMA membership and hydrography evidence table.

Upon extraction of all evidence fields, CAPRM-Flood builds an exposure index table. In doing so, it converts the FEMA/hydrography and terrain evidence tables into normalized 0–100 component scores and applies explicit component weights to produce a countywide relative exposure index with percentile ranking. In total, a staged architecture is utilized in order to provide two forms of defensibility. Each evidence family can be inspected independently before scoring, and both implementation correctness and computational performance are calculated in Python as expected behavior and then compared against C++ outputs. Thus, property-level evidence and preliminary relative indexing are both supported with a clearly defined scope concerning claims made by the system.

IV. IMPLEMENTATION

CAPRM-Flood features a deterministic, reproducible implementation in C++/Python over Monroe County property workloads. Python components perform data retrieval, caching and cache validation, CRS reprojection, trusted reference computations, evidence consolidation and preservation, and overall pipeline coordination. C++ components perform a separate batch spatial computation over fixtures exported by Python as part of pipeline coordination. This allows Python geospatial libraries to maintain a reproducible reference behavior while developing C++ implementations, ultimately yielding a twice-evaluated pair of geometry operations and indexed-search performance whose results can be evaluated for procedural consistency.

A. Property Workload Construction

The property workload comprises all NYS parcel centroid records addressed specifically within Monroe County. Properties have identifying SBL values; these are used to deterministically build workloads by processing identifiers in ascending order, retaining the first nonmissing occurrence of each canonical identifier, and discarding later duplicate identifiers. In total, the workload comprising Monroe County spans 267,362 unique property identifiers; individual workloads taken for development purposes contained subsets of Monroe County’s data of sizes 1K, 10K, and 100K property points. To preserve reproducibility while testing scalability, all workloads were developed via computational fixtures versus random statistical sampling.

B. FEMA Flood Zone Evidence

During the evidence stage, each property centroid is marked based on whether the property lies within a FEMA flood-hazard polygon, and if so, associated flood-zone evidence is recorded. Property points and FEMA National Flood Hazard Layer (NFHL) polygons are reprojected to the shared FEMA topology CRS; a spatial join is subsequently performed to map the two layers together, and a property-level baseline is recorded for each point including property identifier, source and projected coordinates, flood-zone and SFHA membership labels, source geometry identifier, feature index, polygon-match status, and a simplified SFHA Boolean result.

The independent C++ FEMA stage ingests the reprojected property coordinates in addition to Python-flattened polygon-ring fixtures, both of which share a CRS upon ingestion. The polygon-ring table includes polygon parts, ring indices, vertex coordinates, flood-zone and SFHA labels, source geometry identification, and only the feature indices from Python which are necessary to perform a full comparison of computed results. Among all FEMA data extracted, only those FEMA features referenced by the Python baseline implementation are exported to C++ to minimize the functional export size. After ingestion, C++ reconstructs polygon parts and rings, applies bounding-box prefiltering for each FEMA feature, and excludes interior holes to compute property-specific evidence fields for matched polygon status, flood-zone and SFHA label, and feature index.

C. Nearest Water Evidence

Distance from each property’s centroid to the nearest hydrography feature is computed using data from the United States Geological Survey 3D Hydrography Program cache. This cache, bounded by a 20 km buffer around official Monroe County coordinates, includes retained flowline and waterbody feature information. The CRS is maintained as EPSG:26918 to present distances in meters.

A reference implementation of nearest-water computations is performed in Python. This environment loads the hydrography cache, validates feature identifiers and geometry data, normalizes the CRS, constructs a spatial index via Shapely, and computes exact point-to-feature distances. Output records nearest-water distance, along with the identifier, class and type of the selected nearest feature, the source identifiers, name and tie count corresponding to that selected feature, and CRS. Nearest-feature ties are resolved deterministically via canonical feature ID, i.e., selecting the first sorted feature for reported metadata fields while preserving tie count.

Python reference behavior is reproduced in two C++ implementations. First, a brute-force program considers each retained hydrography feature for each property when calculating its nearest feature, computing exact point-to-segment distances for line features and point-to-polygon distances for waterbody features. Points inside of waterbody polygons are considered to be zero meters from the nearest feature; polygon holes are excluded from waterbody interior treatment. The second C++ implementation preserves distance semantics while

implementing a feature-level bounding-volume hierarchy over hydrography feature bounding boxes. In doing so, nodes and features are pruned which cannot improve or tie the current nearest hydrography feature.

D. Terrain Evidence

Terrain data is surveyed via a cached United States Geological Survey 3DEP one-third arc-second DEM. The source raster is reprojected to EPSG:26918 from EPSG:4269, facilitating metric coordinates for local sampling windows, slope calculation, and neighborhood statistics. The reprojected raster is stored with an approximately 8.601 m pixel size. For each property centroid, elevation is sampled; a local mean elevation is computed for a 90 m enclosing raster area; relative elevation is computed as the difference between property elevation and local mean elevation; and slope is computed in degrees using a 3-by-3 Horn-style finite-difference operator. The tabulated terrain output records `property_id`, `terrain_elevation_m`, `terrain_local_mean_elevation_m`, `terrain_relative_elevation_m`, `terrain_slope_degrees`, `terrain_sample_radius_m`, and `terrain_crs`. Terrain evidence is maintained separately from the FEMA/hydrography evidence table for independent inspections of its raster source, checksum, CRS and sampling method.

E. Preliminary Exposure Index

After extracting all evidence fields, CAPRM-Flood builds a preliminary exposure-index table from the FEMA/hydrography and terrain evidence tables. Python performs scoring on this table, currently using the policy version `preliminary_exposure_index_v1`; it produces normalized component scores on a 0–100 scale for FEMA evidence, nearest-water evidence, and terrain evidence. The FEMA component maps flood-zone and SFHA information to component scores. The nearest-water component presents a percentile-based score; shorter nearest-water distances correspond to higher relative exposure. The terrain component combines low absolute elevation percentile and low relative elevation percentile. Finally, the preliminary exposure index is then computed as a weighted sum, for which current weights are 0.40 for the FEMA component, 0.35 for the nearest-water component, and 0.25 for the terrain component. The final index table records `property_id`, the three component scores, `exposure_index_0_100`, `exposure_percentile`, and `scoring_policy_version`.

The scoring stage is intentionally implemented after the lower-level evidence stages rather than inside them, for independent FEMA, water, and terrain evidence analysis before preliminary relative rankings. The resulting index supports countywide screening and comparison.

TABLE I
COUNTYWIDE VALIDATION AND BENCHMARK SUMMARY.

Metric	Countywide result
Property identifiers	267,362
FEMA Python/C++ agreement	267,362/267,362
Brute-force water agreement	267,362/267,362
Indexed water agreement	267,362/267,362
Maximum water distance error	4.658×10^{-10} m
Indexed computation speedup	20.39×
Indexed total-process speedup	19.83×
BVH segment-check reduction	93.34%

F. Reproducibility Artifacts

CAPRM-Flood outputs both intermediate and final artifacts with explicitly-defined paths, schemas, CRS, and validation summaries. Python is used to generate baseline outputs, C++ inputs, comparison against C++ results, benchmark summaries, evidence and index tables, and JSON manifests. The JSON manifests record input paths, I/O checksums, row counts, schema information, CRS metadata, reference implementation benchmarks, and aggregated evidence statistics. Central CAPRM-Flood artifacts include the property cache, Python FEMA baseline, Python nearest-water reference, C++ FEMA output, C++ brute-force nearest-water output, C++ indexed nearest-water output, benchmark records, FEMA and Hydrography evidence table, terrain evidence table, and exposure-index table.

Because the implemented artifact structure records the source and validation context needed to determine how the workload files were produced, each major transformation to the workload becomes auditable, and cached sources can be used for repeatable execution.

V. CURRENT VALIDATION AND RESULTS

CAPRM-Flood currently validates its vector spatial computations by comparing independently computed C++ outputs against outputs computed via GeoPandas/Shapely reference implementation in Python. Property ID matches points across code environments; a full correctness validation requires complete row coverage and categorical agreement across source-feature identifier, CRS, and nearest-water distance agreement within a tolerance of $1e-6$ m. FEMA validation verifies across both implementation outputs the polygon-match status, SFHA status, flood-zone label, and FEMA feature index. Nearest-water validation compares distance, as well as the identifier, class and type corresponding to the nearest hydrography feature, and the source identifiers, feature name, tie count, and distance CRS for each point.

The countywide evidence table contains one row for each Monroe County property identifier sampled. From the sampled properties, 267,358 matched FEMA flood-hazard polygons and four did not. FEMA classification data contained 5,061 SFHA properties and 262,301 non-SFHA properties; observed FEMA zone counts included 262,297 Zone X properties, 4,226 Zone AE properties, 408 Zone A properties, 388 Zone AO properties, 39 Zone VE properties, and four unmatched properties. Regarding nearest-water evidence, 266 properties

had zero distance to a retained hydrography feature, 308 properties had multiple equally nearest features, and 2,934 unique hydrography features were selected as the nearest water feature for at least one property. Median nearest-water distance was 325.346 m, mean distance was 506.411 m, and maximum distance was 2,630.235 m.

Milestone 3 includes the addition of both terrain evidence and a preliminary exposure-index output. Terrain evidence comprises countywide elevation, local mean elevation, relative elevation, slope, sampling radius, and terrain CRS fields for all 267,362 property identifiers. Elevation ranged from 75.000–296.309 m, with mean value 143.197 m and median value 146.828 m. Relative elevation values ranged from -20.669–30.149 m, with mean 0.247 m and median 0.175 m. Mean slope was 1.906 degrees, median slope was 1.234 degrees, and there were no missing countywide slope values; all terrain evidence was reported in EPSG:26918.

FEMA, nearest-water, and terrain components are combined in the preliminary exposure-index stage with current policy `preliminary_exposure_index_v1`. Implemented weights include 0.40 for FEMA evidence, 0.35 for nearest-water evidence, and 0.25 for terrain evidence. The resulting countywide exposure index ranged from 7.915 to 99.929, with a mean of 34.632 and median of 33.728; corresponding countywide exposure percentiles were also recorded. The full Python test suite passed after the terrain and scoring tests were added.

REFERENCES

- [1] National Severe Storms Laboratory, “Severe Weather 101: Flood Basics,” National Oceanic and Atmospheric Administration. [Online]. Available: <https://www.nssl.noaa.gov/education/svrwx101/floods/>. [Accessed: Jun. 9, 2026].
- [2] U.S. Global Change Research Program, *Fifth National Climate Assessment*. Washington, DC, USA: U.S. Global Change Research Program, 2023. [Online]. Available: https://toolkit.climate.gov/sites/default/files/2025-07/NCA5_2023_FullReport.pdf. [Accessed: Jun. 9, 2026].
- [3] Federal Emergency Management Agency, “One inch of floodwater can cause up to \$25,000 of damage in a home.” [Online]. Available: <https://www.fema.gov/vi/print/pdf/node/633873>. [Accessed: Jun. 9, 2026].
- [4] Monroe County, New York, *Hazard Mitigation Plan 2023 Update, Volume I*, Mar. 2023. [Online]. Available: Monroe County Hazard Mitigation Plan. [Accessed: Jun. 9, 2026].
- [5] Federal Emergency Management Agency, “Flood Data Viewers and Geospatial Data: National Flood Hazard Layer.” [Online]. Available: <https://www.fema.gov/flood-maps/national-flood-hazard-layer>. [Accessed: Jun. 9, 2026].
- [6] Federal Emergency Management Agency, “FEMA Flood Map Service Center.” [Online]. Available: <https://msc.fema.gov/portal/home>. [Accessed: Jun. 9, 2026].
- [7] New York State GIS Program Office, “Parcels: New York State Parcels.” [Online]. Available: <https://gis.ny.gov/parcels>. [Accessed: Jun. 9, 2026].
- [8] GeoPandas Developers, “Merging data,” *GeoPandas Documentation*. [Online]. Available: https://geopandas.org/en/stable/docs/user_guide/mergingdata.html. [Accessed: Jun. 9, 2026].
- [9] V. Pandey, A. Kipf, T. Neumann, and A. Kemper, “How good are modern spatial analytics systems?” *Proceedings of the VLDB Endowment*, vol. 11, no. 11, pp. 1661–1673, 2018, doi: 10.14778/3236187.3236213.
- [10] GeoPandas Developers, “geopandas.sjoin,” *GeoPandas Documentation*. [Online]. Available: <https://geopandas.org/en/v1.1.2/docs/reference/api/geopandas.sjoin.html>. [Accessed: Jun. 9, 2026].
- [11] Shapely Developers, “Predicates,” *Shapely Documentation*, ver. 2.1.1. [Online]. Available: <https://shapely.readthedocs.io/en/2.1.1/predicates.html>. [Accessed: Jun. 9, 2026].
- [12] Shapely Developers, “STRtree,” *Shapely Documentation*, ver. 2.1.2. [Online]. Available: <https://shapely.readthedocs.io/en/2.1.2/strtree.html>. [Accessed: Jun. 9, 2026].
- [13] Federal Emergency Management Agency, “National Risk Index for Natural Hazards.” [Online]. Available: <https://www.fema.gov/flood-maps/products-tools/national-risk-index>. [Accessed: Jun. 9, 2026].
- [14] Federal Emergency Management Agency, “Hazus.” [Online]. Available: <https://www.fema.gov/flood-maps/products-tools/hazus>. [Accessed: Jun. 9, 2026].
- [15] Federal Emergency Management Agency, *Flood Assessment Structure Tool*. [Online]. Available: https://www.fema.gov/sites/default/files/documents/fema_flood-assessment-structure-tool.pdf. [Accessed: Jun. 9, 2026].
- [16] M. F. Abdullah, S. Siraj, and R. E. Hodgett, “An overview of multi-criteria decision analysis (MCDA) application in managing water-related disaster events: Analyzing 20 years of literature for flood and drought events,” *Water*, vol. 13, no. 10, Art. no. 1358, 2021, doi: 10.3390/w13101358.
- [17] H. Allafta and C. Opp, “GIS-based multi-criteria analysis for flood prone areas mapping in the trans-boundary Shatt Al-Arab basin, Iraq-Iran,” *Geomatics, Natural Hazards and Risk*, vol. 12, no. 1, pp. 2087–2116, 2021, doi: 10.1080/19475705.2021.1955755.
- [18] S. Chengu, M. Assen, and E. Gebeyehu, “Multi-criteria decision analysis for flood hazard mapping in the Itang watershed, lower Baro-Akobo Basin, Southwest Ethiopia,” *Discover Environment*, vol. 3, Art. no. 110, 2025, doi: 10.1007/s44274-025-00289-5.
- [19] Boost C++ Libraries, “R-tree quick start,” *Boost.Geometry Documentation*. [Online]. Available: https://www.boost.org/doc/libs/latest/libs/geometry/doc/html/geometry/spatial_indexes/rtree_quickstart.html. [Accessed: Jun. 9, 2026].